Verification and Validation Report: SFWRENG 4G06 - Capstone Design Process

Team 17, DomainX

Awurama Nyarko
Haniye Hamidizadeh
Fei Xie
Ghena Hatoum

March 9, 2026

1 Revision History

Date	Version	Notes
March 6	Rev 1	First copy of VnV Report done by the whole team

2 Symbols, Abbreviations and Acronyms

Symbol	Description
AHP	Analytic Hierarchy Process
API	Application Programming Interface
CR	Consistency Ratio (AHP)
DB	Database
FR	Functional Requirement
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping (e.g., Django ORM)
SRS	Software Requirements Specification
T	Test Case Identifier
VnV	Verification and Validation
JSON	JavaScript Object Notation
SRS	Software Requirements Specification
UUID	Universally Unique Identifier
PATCH / POST / PUT	Standard HTTP methods used for updating, creating, and replacing r

All additional terms conform to those defined in the SRS Glossary (Section 4.1).

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**
- 3.1 User Authentication and Profile Management 1
- 3.2 Domain and Data Management 3
- 3.3 Visualization and Reporting 6
- 3.4 Features Currently in Progress 8

4 Nonfunctional Requirements Evaluation **10**
- 4.1 Performance & Scalability Experiments 10
  - 4.1.1 Test Case Summary (T–NF1: Performance and Scalability) 10
  - 4.1.2 Method (Rev0) 11
  - 4.1.3 Results 11
  - 4.1.4 Analysis 12
  - 4.1.5 Limitations and Next Steps 13
- 4.2 Reliability and Fault Tolerance 13
  - 4.2.1 Test Case Summary (T–NF2: Reliability and Fault Tolerance) 14
  - 4.2.2 Results 15
  - 4.2.3 Analysis 15
  - 4.2.4 Limitations and Next Steps 16
- 4.3 Usability 16
  - 4.3.1 Test Case Summary (T–NF3: Usability Pilot) 17
  - 4.3.2 Method (Rev0 Pilot) 17
  - 4.3.3 Results 18
  - 4.3.4 Analysis and Traceability 19
  - 4.3.5 Limitations and Next Steps 19
- 4.4 Computational Accuracy 20
  - 4.4.1 Test Case Summary (T–NF4: Computational Accuracy) 20
  - 4.4.2 Method (Rev0) 20
  - 4.4.3 Results 20
  - 4.4.4 Analysis and Discussion 21
  - 4.4.5 Limitations and Next Steps 21

4.5 Security and Access Control 23
4.5.1 Test Case Summary (T–NF5: Security and Access Control) 24
4.5.2 Results 25
4.5.3 Analysis 25
4.5.4 Limitations and Next Steps 26
4.6 Maintainability 26
4.6.1 Test Case Summary (T–NF6: Maintainability) 27
4.6.2 Results 27
4.6.3 Analysis and Traceability 28
4.6.4 Limitations and Next Steps 29

5 Unit Testing **29**
5.1 Data Edit Module 29
5.2 User Management Module 32
5.3 Automated Metrics Module 34
5.4 Domains Page Module 39
5.5 Comparison Module 41
5.6 Visualization Module 42
5.7 Database Persistence Module 43

6 Changes Due to Testing **49**
6.1 AHP Methodology Alignment 49
6.2 Visualization and Data Density 49
6.3 Data Integrity and Input Validation 49

7 Automated Testing **50**

8 Trace to Requirements **50**

9 Trace to Requirements (Nonfunctional) **51**

10 Trace to Modules **53**

11 Code Coverage Metrics **53**

List of Tables

1 T–NF1 performance experiment: test case summary (Rev0). . 10
2 T–NF1 response time results (Rev0). 12
3 T–NF2 reliability and fault tolerance: test case summary (Rev0). 14
4 Reliability experiment results (Rev0). 15
5 T–NF3 usability pilot: test case summary (Rev0). 17
6 Pilot usability task results (n = 1 proxy user). 18
7 Usability traceability summary (Rev0 pilot). 19
8 T–NF4 computational accuracy: test case summary (Rev0). . 21
9 Computed AHP priorities recorded from the tool (Rev0). . . . 22
10 T–NF5 security and access control: test case summary (Rev0). 24
11 Security and access control results (Rev0). 25
12 T–NF6 maintainability: test case summary (Rev0). 27
13 Maintainability evidence summary (Rev0). 28
14 Traceability Between Requirements and System Test Cases . . 51
15 Nonfunctional Requirements Traceability Matrix 52
16 Traceability Between Modules and System Test Cases 53
17 Backend Code Coverage Report 54

List of Figures

This document ...

3 Functional Requirements Evaluation

This section presents the results of the functional tests defined in the project's Verification and Validation (VnV) Plan. These tests verify that the implemented system satisfies the functional requirements specified in the Software Requirements Specification (SRS). The tests are organized according to the system's main feature areas.

3.1 User Authentication and Profile Management

The test results below verify that users can successfully authenticate and manage their account information within the system. These tests ensure that login, profile updates, and password modification operate correctly while restricting access to authorized users only.

Test Case Identifier: T–AC–1

Input: Submission of valid sign-up information through the system interface.

Expected Output: The expected result is that a new user account is created successfully and the user is redirected to the login page.

Actual Output: The actual result is that a new user account is created successfully and the user is redirected to the login page.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR2

Test Case Identifier: T–AC–2

Input: Submission of valid login credentials through the system login interface.

Expected Output: The expected result is that the user enters valid login credentials and is redirected to the system dashboard.

Actual Output: The actual result is that the user enters valid login credentials and is redirected to the system dashboard.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR3

Test Case Identifier: T–AC–3

Input: A user with the Contributor role logs into the system and attempts to edit metric values in the interactive data table.

Expected Output: The expected result is that the user is able to view library data and successfully edit metric values in the interactive table.

Actual Output: The actual result is that the user is able to view library data and successfully edit metric values in the interactive table.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): SR–AC2

Test Case Identifier: T–AC–4

Input: A user with the Super Admin role logs into the system and attempts to access administrative functionality for managing users.

Expected Output: The expected result is that the user is able to access administrative functionality and manage user accounts within the system.

Actual Output: The actual result is that the user is able to access administrative functionality and manage user accounts within the system.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): SR–AC2

Test Case Identifier: T–AC–5

Input: Submission of invalid login credentials or an unregistered email through the system login interface.

Expected Output: The expected result is that access is denied, an Invalid credentials error message is displayed, and no session token is created.

Actual Output: The actual result is that access is denied, an Invalid credentials error message is displayed, and no session token is created.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): SR–AC5, UH–UP3, LF–AR6

Test Case Identifier: T–AC–6

Input: A Super Admin updates the role of an existing user (e.g., Contributor to Super Admin) through the user management interface.

Expected Output: The expected result is that the user role is updated successfully and the interface updates to reflect the new permissions associated with the role.

Actual Output: The actual result is that the user role is updated successfully and the interface updates to reflect the new permissions associated with the role.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): SR–AC4, PR–SC3, FR7

Test Case Identifier: T–AC–7

Input: An authenticated user accesses their profile page and submits a request to change their password using the password update form.

Expected Output: The expected result is that the user password is updated successfully and the new password is required for subsequent logins.

Actual Output: The actual result is that the user password is updated successfully and the new password is required for subsequent logins.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): SR–AC5, PR–SC1

3.2 Domain and Data Management

The following tests verify that domains, libraries, and metrics can be created, modified, and stored correctly within the system. These tests ensure that domain data management and metric updates function as expected.

Test Case Identifier: T–DM–1

Input: A Super Admin submits valid domain information through the domain creation interface.

Expected Output: The expected result is that the new domain is created successfully, appears in the system interface, and is stored in the database with a unique identifier.

Actual Output: The actual result is that the new domain is created successfully, appears in the system interface, and is stored in the database with a unique identifier.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR–5, SR–INT1, SR–AC3

Test Case Identifier: T–DM–2

Input: A Contributor or Super Admin adds a library to a domain by providing a valid GitHub repository URL.

Expected Output: The expected result is that the library is added successfully and repository metadata (e.g., commits, issues, lines of code, etc.) is automatically retrieved using GitHub API, SCC, and GitStats without errors.

Actual Output: The actual result is that the library is added successfully and repository metadata (e.g., commits, issues, lines of code, etc.) is automatically retrieved using GitHub API, SCC, and GitStats without errors.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR–7, FR–10, OE–IA1, SR–INT2, SR–INT3

Test Case Identifier: T–DM–3

Input: A Contributor or Super Admin edits a metric value for a library through the metric editing interface.

Expected Output: The expected result is that the updated metric value is stored successfully in the database and the AHP ranking is

recalculated to reflect the new value.

Actual Output: The actual result is that the updated metric value is stored successfully in the database and the AHP ranking is recalculated to reflect the new value.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR–7, FR–12, SR–INT4, SR–IM2

Test Case Identifier: T–DM–4

Input: An API request for repository analysis is forced to fail by temporarily reducing the request timeout during testing.

Expected Output: The expected result is that the system reports the analysis failure to the user while preserving previously stored metric data for the library.

Actual Output: The actual result is that the system displays a failure message in the interface while previously stored metric data remains unchanged.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): PR–RFT1, PR–RFT2, SR–INT5

Test Case Identifier: T–DM–5

Input: A user selects the export option to download the comparison table for a domain as an Excel file.

Expected Output: The expected result is that the system generates and downloads an Excel file containing all libraries in the selected domain along with their associated metrics and stored metric values.

Actual Output: The actual result is that the system generates and downloads an Excel file containing all libraries in the selected domain along with their associated metrics and stored metric values.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR–14, SR–INT7, OE–PR4

3.3 Visualization and Reporting

The following tests verify that the system correctly generates visualizations for domain comparison and accurately represents stored metric values for libraries.

Test Case Identifier: T–VR–1

Input: A user opens the visualization page for a domain containing at least three libraries, selects one or more metrics and libraries, and clicks the *Visualize* button.

Expected Output: The expected result is that the system generates comparative graphs for the selected libraries and metrics on the visualization page.

Actual Output: The actual result is that the system generates comparative graphs for the selected libraries and metrics on the visualization page.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR–13, FR–12, SR–VIS1, SR–IM1

Test Case Identifier: T–VR–2

Input: A user modifies the selected metrics or libraries on the visualization page and clicks the *Visualize* button to regenerate the graphs.

Expected Output: The expected result is that the system updates the graphs to reflect the newly selected metrics or libraries.

Actual Output: The actual result is that the system updates the graphs to reflect the newly selected metrics or libraries.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR–13, SR–VIS2, PR–SL1

Test Case Identifier: T–VR–3

Input: A user clicks the *Download All* button on the visualization page after generating graphs.

Expected Output: The expected result is that the system generates and downloads the visualization plots as PNG image files.

Actual Output: The actual result is that the system generates and downloads the visualization plots as PNG image files.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR–15, OE–PR4, SR–INT7

Test Case Identifier: T–VR–4

Input: A user resizes the browser window or opens the visualization page on different screen resolutions.

Expected Output: The expected result is that the graphs remain readable and the interface layout adjusts correctly without overlapping elements.

Actual Output: The actual result is that the graphs remain readable and the interface layout adjusts correctly without overlapping elements.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): LF–AR2, LF–SR2, UH–AR1

Test Case Identifier: T–VR–5

Input: A domain contains a library where one or more metric values are missing when the user generates visualization graphs.

Expected Output: The expected result is that the visualization is generated successfully and the system handles the missing metric value without runtime errors.

Actual Output: The actual result is that the visualization is generated successfully and the system handles the missing metric value without runtime errors.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): PR–RFT1, SR–INT5

Test Case Identifier: T–VR–6

Input: A user opens the Manage Metrics page for a domain.

Expected Output: The expected result is that the system displays a list of all metrics associated with the domain, including their names and corresponding definitions or configuration details.

Actual Output: The actual result is that the system displays a list of all metrics associated with the domain, including their names and corresponding definitions or configuration details.

Expected and Actual Output Match: True

Relevant Functional Requirement(s): FR–11, UH–LR2, LF–AR3

3.4 Features Currently in Progress

Some functional requirements identified in the SRS are still under development, while others were revised after further discussion and feedback. These items are not yet fully implemented or were removed from the final project scope. Their current status is summarized below.

SR–AC1: Public Viewing and Restricted Editing

This requirement is currently under development as part of the system's authorization implementation. The intended behaviour is to allow public users to view published domain data while restricting modification privileges to approved users only. Page-level and button-level authorization controls have been partially implemented and are still being refined.

FR–2: User Self Sign-Up

This requirement was originally intended to allow users to create their own accounts through a public sign-up page. During development, this design was reconsidered, and the system is being revised so that only a Super Admin can create new user accounts. As a result, self sign-up will not appear in the final implementation.

SR–AC6, SR–AU1, SR–IM1: Unauthorized API Access Control

Unauthorized API access control is currently under development. The intended behaviour is that attempts to perform restricted actions, such as deleting a domain without sufficient privileges, will return an HTTP 403 Forbidden response and generate an audit log entry. At present, restricted actions are hidden in the user interface, while API-level authorization checks are still being implemented and are not yet ready for formal testing.

Security requirements related to SQL injection prevention and secure credential storage are already satisfied through the use of the Django ORM, which avoids raw SQL queries, and environment variables for sensitive configuration data.

FR–4: Password Reset via Email

This feature is intended to allow users to reset their password through an email verification workflow. The expected behaviour is that a user who forgets their password receives a reset email, verifies their identity through a tokenized link, and sets a new password for future logins.

This functionality is currently under development and has not yet been fully implemented or tested.

FR–9: Bulk Data Upload

This requirement was originally intended to support bulk upload of domain data through an Excel template. Based on supervisor feedback, this functionality was removed from the project scope and will not be implemented. Therefore, no testing was performed for this requirement.

Relevant Functional Requirement(s): FR–9, SR–INT1, UH–EU2

FR–6: Publish Domain for Public Access

This feature is currently under development as part of the remaining authorization work. The intended behaviour is that a completed domain can be published and made visible to all users, with its data available for public viewing and download. This functionality will be completed together with the remaining access-control features.

Relevant Functional Requirement(s): FR–6, OE–PR3, FR–14

4 Nonfunctional Requirements Evaluation

This section reports verification results for the system's nonfunctional requirements (NFRs). Each evaluation includes explicit inputs and outputs, an expected-versus-actual comparison, and traceability back to the relevant NFR identifiers. Tables are used to summarize results and to support reproducibility.

4.1 Performance & Scalability Experiments

This subsection reports Revision 0 results for the performance and scalability experiment specified in the VnV Plan (§4.2.1). The experiment measures response times for dashboard load, filtering, AHP recomputation, and export, and summarizes median and 95th-percentile (p95) values.

4.1.1 Test Case Summary (T–NF1: Performance and Scalability)

Table 1: T–NF1 performance experiment: test case summary (Rev0).

Test Case ID	T–NF1 (Performance and Scalability)
Traceability	PR-SL1 (Speed and Latency Requirement).
Initial State	System deployed and accessible; existing domains available with known library counts (2 and 10).
Input	Record time to (1) load the dashboard and render the ranking graph, (2) apply a domain filter, (3) update category weights to trigger AHP recomputation, and (4) export visualizations.
Expected Output	Median and p95 response times are recorded; response times remain within a few seconds for interactive use.
Actual Output	Dashboard load and AHP recomputation completed within a few seconds. Domain filtering did not narrow results (no observable effect). Export appeared instantaneous in repeated trials.
Expected vs. Actual Match	Partial (latency acceptable for measured actions; filtering defect prevented filter timing; scaling evaluated at 2 and 10 libraries only).

4.1.2 Method (Rev0)

Timings were collected manually using a stopwatch, measured from the moment the action was initiated (click) to the moment the interface reached a stable state (graph rendered or updated). Due to manual timing, values include human reaction time and should be treated as conservative upper bounds. For small sample sizes, p95 is reported conservatively as the maximum observed value.

4.1.3 Results

Table 2 summarizes the observed median and p95 response times.

Table 2: T–NF1 response time results (Rev0).

Action	**Median (s)**	**p95 (s)**	**Notes**
Dashboard load (2 libraries, n=8)	0.55	0.74	Time to render ranking view.
Dashboard load (10 libraries, n=10)	2.65	2.92	Time to render ranking view.
Recompute AHP (edit category weights, n=10)	3.61	3.90	Time from save/apply to updated view.
Apply domain filter	N/A	N/A	Filter input produced no narrowing of results.
Export visualizations	< 0.5	< 0.5	Appeared instantaneous (below stopwatch resolution).

4.1.4 Analysis

Across the measured actions, response times were within a few seconds, with worst observed values below 4 seconds. Dashboard load increased from a median of 0.55 seconds (2 libraries) to a median of 2.65 seconds (10 libraries), indicating higher latency as library count increases in the tested configuration. AHP recomputation completed in under 4 seconds in all trials, which is acceptable for interactive use.

Domain filtering produced no observable narrowing of the domain list. This is treated as a functional defect and prevented measurement of filter latency in Revision 0.

4.1.5 Limitations and Next Steps

Revision 0 results include only 2- and 10-library datasets and manual timing. In the next revision, performance will be re-measured using the planned dataset sizes (5, 10, 20, and 40 libraries) and automated timing collection where possible. The domain filtering defect will be addressed and re-tested to capture filter latency.

4.2 Reliability and Fault Tolerance

This subsection reports Revision 0 results for reliability and fault tolerance. The goal is to confirm the system remains stable, consistent, and capable of recovery when services fail or operations are interrupted.

4.2.1 Test Case Summary (T–NF2: Reliability and Fault Tolerance)

Table 3: T–NF2 reliability and fault tolerance: test case summary (Rev0).

Test Case ID	T–NF2 (Reliability and Fault Tolerance)
Traceability	PR-RFT1–PR-RFT3, SR-INT5.
Initial State	System deployed and accessible; at least one existing domain is loaded.
Input	(1) Simulate service/API failure by forcing the client offline in the browser and attempt common read actions (switch view tabs and open the comparison tool). (2) Interrupt a mid-update transaction by modifying category weights, clicking save, and toggling offline immediately after the save attempt. (3) Restore connectivity and verify recovery and data integrity.
Expected Output	The tool remains stable (no crash), displays a user-visible warning/error instead of failing silently, and recovers once services return. Interrupted updates rollback to the last valid state and do not leave partial/corrupt state.
Actual Output	When offline, the interface remained responsive and did not crash. Domain metadata and results were not displayed (Details panel showed Name: N/A) and key actions produced no observable effect. No user-visible warning or cached fallback results were observed. After restoring connectivity (No throttling), the dashboard resumed normal operation and loaded domain details and rankings correctly. In the interrupted-write test (save category weights then go offline), the system reverted to the pre-change configuration after recovery (the attempted change was not persisted), and no half-created/partial state was observed.
Expected vs. Actual Match	Partial (stable, recoverable, and rollback-safe, but weak failure messaging and no offline fallback behavior).

4.2.2 Results

Table 4: Reliability experiment results (Rev0).

Scenario	Result	Notes
Client forced offline (API failure proxy)	Partial	No crash; content unavailable (Name: N/A); actions no-op; no warning shown; no cached fallback observed.
Recovery after reconnect	Pass	After returning online, domain details and rankings rendered normally.
Interrupted mid-update transaction (save then offline)	Pass	After recovery the system reverted to the last valid state (changes not persisted); no partial state observed.
Integrity after failure/recovery	Pass	No half-created/corrupt state observed in the UI after recovery.
HTTP 429 (rate limiting)	Not tested	Not reproducible/observed in Rev0.
Backup and restore (within one hour)	Pending	Requires DB-level procedure execution and timing.

4.2.3 Analysis

The system remained stable during a simulated service outage (offline mode) and successfully recovered once connectivity was restored. The interrupted-write test also indicates safe failure handling: the system returned to the last valid configuration after recovery and did not leave partial or corrupt state.

However, the failure mode is not communicated clearly to the user: data

becomes unavailable and actions become no-ops without a warning message or fallback behavior. Improvements should include explicit error messaging, disabling actions that require connectivity, and (if feasible) displaying cached last-known results with an "offline" banner. Save operations should also provide clear success/failure feedback when the backend is unreachable.

4.2.4 Limitations and Next Steps

Revision 0 reliability evidence is based on manual fault injection (offline mode) and a single interrupted-write scenario. In the next revision, explicit HTTP 429 scenarios will be exercised if rate limiting is enabled, and a backup-and-restore procedure will be executed and timed to confirm recovery within one hour.

Conclusion: Rev0 demonstrates stability and safe rollback under outages, but highlights a gap in user-facing fault messaging and offline fallback behavior.

4.3 Usability

This subsection reports Revision 0 results for the usability evaluation (T–NF3). The VnV Plan defines a one-to-five usability scale where higher values indicate better usability and uses a mean rating ≥ 4.0 as the success criterion. For Revision 0, available evidence consists of a pilot task-based proxy evaluation (n = 1), reported below as preliminary evidence.

4.3.1 Test Case Summary (T–NF3: Usability Pilot)

Table 5: T–NF3 usability pilot: test case summary (Rev0).

Test Case ID	T–NF3 (Usability)
Initial State	System deployed and accessible; tester starts unauthenticated; no pre-existing domain required.
Input	Tester completes eight core tasks (account creation/login, create domain, add metrics, add libraries, fill manual metrics, export data, visualize and export raw metrics, visualize and export AHP results) and records task ratings plus qualitative comments.
Expected Output	Core tasks are discoverable and completable without developer assistance; charts and exports are understandable; required inputs are supported by clear feedback and error messaging (saving does not fail silently).
Actual Output	All eight tasks were completed. Two tasks showed reduced ease due to discoverability and feedback issues: adding metrics and locating manual metric fields.
Expected vs. Actual Match	Partial (workflow completion succeeded; specific usability issues were identified).

4.3.2 Method (Rev0 Pilot)

A proxy user (Ryan) completed a structured, asynchronous worksheet to reduce developer influence. The worksheet captured task *difficulty* on a one-to-five scale where 1 = Very Easy and 5 = Very Hard.

Because the VnV Plan convention is higher = better usability, we also report a converted task *ease* score using:

$$\text{EaseScore} = 6 - \text{DifficultyScore}.$$

This conversion preserves the raw ratings while aligning the pilot results with the plan's direction.

4.3.3 Results

Table 6 summarizes the proxy user's task ratings and key observations. Both the raw difficulty score and the converted ease score are reported.

Table 6: Pilot usability task results (n = 1 proxy user).

Task	**Diff.**	**Ease**	**Key observation**
Create an account and login	1	5	No email confirmation after registration.
Create a new domain	1	5	Straightforward.
Add metrics to the domain	3	3	Entry point not obvious; saving blocked without a clear warning.
Add two libraries to the domain	1	5	Easy to complete.
Fill out manually added metrics	3	3	Difficult to locate fields due to dense inputs and scrolling.
Export gathered data as a CSV	1	5	Easy export.
Visualize and export raw metric values	1	5	Easy to use; visualization perceived as useful.
Visualize and export AHP calculations	1	5	Easy to find and download.

Summary statistics for the pilot:

- Mean task difficulty: 3/2 (1.5); median difficulty: 1.
- Mean converted ease: 9/2 (4.5); median converted ease: 5.
- Lowest ease tasks (converted): add metrics; fill manual metrics (3/5).

4.3.4 Analysis and Traceability

The converted ease mean of 4.5/5.0 suggests the workflow is generally usable in the pilot. However, two targeted usability risks were identified: (1) discoverability and feedback during metric creation, and (2) navigational friction when locating manual metric fields among dense inputs. These issues map directly to the traceability targets for clear feedback and efficient navigation.

Table 7 summarizes traceability between usability requirements and pilot evidence.

Table 7: Usability traceability summary (Rev0 pilot).

Req. ID	**Intent**	**Pilot evidence**
UH-UP3	Clear feedback and error messages	Saving was blocked during metric creation without a clear warning, increasing confusion and trial-and-error.
UH-EU1	Efficient navigation for core actions	Manual metric fields were difficult to locate among many inputs, which suggests increased clicks and time-on-task.
LF-SR2	Clear, minimalist graph design	Visualization and export of raw metrics and AHP results were rated very easy and described as useful.
LF-AR2	Responsiveness across screens	Not explicitly evaluated in the Rev0 pilot; pending in the full study.
UH-AR1	Font scalability/accessibility	Not explicitly evaluated in the Rev0 pilot; pending in the full study.

Additional improvement opportunities were noted by the proxy user, including providing registration confirmation feedback and supporting bulk import of libraries and automatic suggestion of repository URLs to reduce manual effort.

4.3.5 Limitations and Next Steps

The Revision 0 usability evidence is a pilot result with a single proxy user. The Appendix B usability survey with multiple participants will be executed

in the next revision to evaluate the plan's survey-based success criterion. The pilot findings above will be converted into tracked usability issues and re-tested after implementing improved discoverability for metric creation and clearer save-time feedback for required fields.
Evidence: Pilot usability worksheet (Ryan), stored in the project repository at `/test/reports/usability/UserTesting-Ryan.pdf`.

4.4 Computational Accuracy

This subsection reports Revision 0 results for the computational-accuracy evaluation (T–NF4). The purpose is to confirm that (1) AHP category priorities produced by the tool are stable and consistent with the configured pairwise comparisons, and (2) a spot check of auto-filled repository fields matches externally verifiable values (GitHub).

4.4.1 Test Case Summary (T–NF4: Computational Accuracy)

4.4.2 Method (Rev0)

Two complementary checks were performed.

AHP priority computation (tool output capture). A category-weight configuration was entered in the AHP interface. The *Calculated Priorities* panel was used to record the resulting priority percentages exactly as displayed. The consistency ratio (CR) value shown by the tool UI was also recorded.

Repository-field spot check (external verification). For a small sample of libraries, a key automatically populated repository field (e.g., last commit date) was compared against the corresponding value shown on GitHub.

4.4.3 Results

AHP priorities produced by the tool. Table 9 lists the computed priorities recorded from the tool.

During this run, the tool displayed the consistency ratio (CR) as `NaN%` in the UI while still producing the priority vector above.

Table 8: T–NF4 computational accuracy: test case summary (Rev0).

Test Case ID	T–NF4 (Computational Accuracy)
Initial State	User is authenticated; a domain exists with at least two libraries; the Category Weights (AHP) configuration and repo-metric views are accessible.
Input	(1) Configure AHP pairwise comparisons and record the computed category priorities and consistency indicator shown by the tool. (2) Select a small sample of libraries and cross-check key auto-filled repository fields against GitHub (e.g., last commit date).
Expected Output	Tool computes and displays a stable priority vector consistent with the configured comparisons. Auto-filled repository fields match GitHub for the sampled libraries, or discrepancies are identified for correction.
Actual Output	AHP priorities were computed and recorded (Table 9). For the sampled libraries, the checked repository field(s) matched GitHub (spot check).
Expected vs. Actual Match	Pass for observed checks (Rev0), with additional quantitative comparisons planned (see limitations).

Repository-field spot check. For the sampled libraries, the checked repository field(s) (e.g., last commit date) matched GitHub during the manual cross-check.

4.4.4 Analysis and Discussion

The recorded priority vector reflects the intended ordering in which *Installability*, *Quality*, and *Correctness and Verifiability* receive higher weights than the remaining categories, which are weighted equally in this configuration.

The repository-field spot check provides preliminary evidence that the auto-filled repository values are consistent with GitHub for the sampled cases.

4.4.5 Limitations and Next Steps

Revision 0 evidence is limited to (1) capturing the tool's computed priorities from a single configuration and (2) a small manual spot check of repository

Table 9: Computed AHP priorities recorded from the tool (Rev0).

Category	**Priority**
Installability	25.6%
Quality	15.1%
Correctness and Verifiability	9.6%
Surface Reliability	5.0%
Surface Robustness	5.0%
Surface Usability	5.0%
Reusability	5.0%
Visibility/Transparency	5.0%
Surface Understandability	5.0%
Raw Metrics (Measured via git_stats)	5.0%
Raw Metrics (Measured via scc)	5.0%
Repo Metrics (Measured via GitHub)	5.0%
Uncategorized	5.0%

fields. The full VnV Plan method for T–NF4 includes manual AHP computation and relative-error checks with a 2% threshold; these quantitative checks are planned for the next revision. Specifically, we will:

- Prepare a small AHP matrix and compute expected pairwise weights manually (or using a trusted reference implementation), then run the same inputs through the tool and compare priorities.
- Calculate relative error (%) for each compared value and flag any deviation greater than 2%.
- Expand the repository-field verification to additional fields (e.g., repository creation date and last commit date) across a larger library sample.
- Investigate the `NaN%` CR display to determine whether it is an expected UI behavior for the given comparison pattern or a defect in the consistency calculation display.

Traceability: PR-PA1 (auto-filled data accuracy) and FR12 (AHP ranking computation).

4.5 Security and Access Control

This subsection reports Revision 0 results for security and access control. The goal is to validate that credentials and permissions are handled securely and that unauthorized actions are prevented.

4.5.1 Test Case Summary (T–NF5: Security and Access Control)

Table 10: T–NF5 security and access control: test case summary (Rev0).

Test Case ID	T–NF5 (Security and Access Control)
Traceability	PR-SC1–PR-SC3, SR-AC6, SR-IM1, SR-AU1.
Initial State	System deployed and accessible; both non-privileged and privileged user accounts available.
Input	(1) Static review of backend authentication configuration and security settings. (2) Dynamic check of restricted UI features using a non-privileged account.
Expected Output	Authentication uses secure mechanisms (hashed password storage, validated password policy, secure cookies in production). Unauthorized actions are blocked and do not modify system state.
Actual Output	Static inspection shows a custom user model (`AUTH_USER_MODEL = users.CustomUser`) and DRF authentication via `users.auth.CookieJWTAuthentication`. Password validation is enabled using Django's built-in validators (similarity, minimum length, common password, numeric password checks). Deployment security settings use environment-based configuration for `SECRET_KEY`, and enable `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, and `SECURE_SSL_REDIRECT` when not in debug/local mode. Backend logging is configured via `LOGGING` with console output and enabled loggers for `django`, `celery`, and `api`. Dynamic testing with a non-privileged user showed restricted admin features are hidden in the UI and were not accessible via normal navigation.
Expected vs. Actual Match	Partial (authentication, logging configuration, and access control evidenced; lockout and session timeout behavior not fully verified in Rev0).

4.5.2 Results

Table 11: Security and access control results (Rev0).

Scenario	Result	Notes
Authentication mechanism review	Pass	DRF uses `CookieJWTAuthentication`; custom user model configured.
Password policy validation review	Pass	`AUTH_PASSWORD_VALIDATORS` enabled with Django validators.
Production cookie / HTTPS settings review	Pass	Secure cookies and SSL redirect enabled when not debug/local.
Unauthorized feature access (UI)	Pass	Admin-only buttons hidden for non-privileged user.
Server logging configuration review	Pass	`LOGGING` configured with console handler; `django`, `celery`, and `api` loggers enabled (INFO/DEBUG).
Session timeout / lockout verification	Pending	`SESSION_COOKIE_AGE` not configured explicitly; lockout not verified in Rev0.

4.5.3 Analysis

Revision 0 evidence indicates the system uses framework-aligned security practices: password validation is enabled through Django's built-in validators, authentication is enforced through a cookie-based JWT mechanism, and production deployments enable secure cookies and HTTPS redirection. Access control is also supported through UI-level restriction of admin-only functionality for non-privileged users.

Server logging is configured via the Django `LOGGING` setting with a console handler; however, persistence/retention as a long-term audit trail was not verified in Revision 0.

The main remaining gap for Rev0 is confirmation of session timeout and account lockout behavior. These should be validated in the next revision through explicit tests (e.g., unauthorized route attempts observed as 401/403 and confirmation of logging retention, plus lockout/timeout configuration where required).

4.5.4 Limitations and Next Steps

Next steps include (1) verifying unauthorized route access returns 401/403 and does not modify state, (2) confirming whether security-relevant events are persisted beyond console output (e.g., centralized log retention or audit tables), and (3) explicitly configuring and testing session timeout and lockout behavior if required by the VnV Plan.
Evidence: Backend configuration in `src/backend/DomainX/settings.py` (`AUTH_USER_MODEL`, `REST_FRAMEWORK`, `AUTH_PASSWORD_VALIDATORS`, deployment secure-cookie/HTTPS settings, and `LOGGING`); authentication module `users.auth.CookieJWTAuthentication`; UI observation that admin-only buttons are hidden for non-privileged users.

4.6 Maintainability

This subsection reports Revision 0 results for maintainability. Evidence is taken from automated GitHub Actions runs and repository documentation related to testing. The evaluation targets MS-MR1–MS-MR6 (maintainability standards: style compliance, modularity, testability, and documentation).

4.6.1 Test Case Summary (T–NF6: Maintainability)

Table 12: T–NF6 maintainability: test case summary (Rev0).

Test Case ID	T–NF6 (Maintainability)
Initial State	Repository contains an automated test workflow in GitHub Actions and a dedicated `test/` directory with documentation.
Input	Trigger the `tests.yml` workflow via a pull request update and inspect the run status, logs, and artifacts. Inspect `test/` documentation to confirm test execution instructions are provided.
Expected Output	Automated tests execute successfully; a CI coverage summary is produced and meets the target (planned $\geq 80\%$); test documentation exists for developers to run tests locally.
Actual Output	The `tests.yml` workflow executed successfully with separate `backend-test` and `frontend-test` jobs. Backend tests ran with coverage and produced TOTAL backend coverage of 73%. A coverage HTML artifact (`coverage-html`) was uploaded. The repository includes `test/README.md` documenting how to run tests locally.
Expected vs. Actual Match	Partial (tests and reporting present; coverage below 80% target; documentation present).

4.6.2 Results

Table 13 summarizes maintainability evidence for Rev0. The workflow run selected for evidence is the most recent successful run at the time of reporting.

Table 13: Maintainability evidence summary (Rev0).

Item	Observed	Notes
Automated CI tests (GitHub Actions)	Yes	`tests.yml` runs on pull requests with backend and frontend jobs.
Backend tests	Pass	`pytest` executed in CI; tests passed (with warnings).
Frontend tests	Pass	Frontend job executed in CI (Playwright-based).
Coverage reporting enabled	Yes	Coverage printed in logs; HTML report uploaded as artifact.
Overall backend coverage	73%	Below planned target of $\geq$ 80%.
Test execution documentation	Yes	`test/README.md` documents setup and running `pytest`.
Lint/format automation	Not observed	No CI evidence of flake8/black in the sampled workflow logs.

4.6.3 Analysis and Traceability

The existence of an automated test workflow integrated into pull requests supports maintainability through testability and regression resistance. The workflow runs backend and frontend tests and produces a coverage report, which enables repeatable inspection of covered and uncovered code paths. The presence of `test/README.md` further supports maintainability by documenting how developers can run the test suite locally.

The observed overall backend coverage (73%) does not meet the planned $\geq$ 80% target for Rev0. This indicates that additional tests should be added, particularly for modules with low coverage, to increase confidence in future refactors and to align with MS-MR testability expectations.

This evaluation traces to MS-MR1–MS-MR6. Rev0 evidence most directly supports testability and documentation. Explicit evidence for style compliance via automated lint/format checks remains to be captured.

4.6.4 Limitations and Next Steps

In the next revision, maintainability will be strengthened by: (1) increasing automated test coverage to reach the $\geq 80\%$ target, (2) adding or capturing explicit lint/format checks in CI if required by the VnV Plan, and (3) recording a brief team code walkthrough note summarizing modular structure and documentation completeness, with follow-up items tracked in GitHub issues. **Evidence:** GitHub Actions `tests.yml` workflow and latest successful run (`backend-test` and `frontend-test`), including coverage output (TOTAL coverage) and the `coverage-html` artifact; repository test documentation at `/test/README.md`.

5 Unit Testing

The following unit tests concern behaviour-hiding modules that are implemented by the tool. Modules implemented by external libraries are assumed to be tested themselves and therefore don't need to be unit tested.

5.1 Data Edit Module

1. `test_list_libraries_returns_only_domain_libraries`
   Initial State: Domain "D1" with 2 libraries, another domain "Other" with 1 library
   Input: GET request to `/api/libraries/by_domain/{domain_ID}/`
   Expected Output: Returns only libraries belonging to the specified domain
   Actual Output: Returned libraries "A" and "B" from domain "D1"
   Result: Pass

2. `test_create_library_requires_domain`
   Initial State: N/A
   Input: POST request to `/api/libraries/` with empty payload
   Expected Output: HTTP 400 Bad Request; domain is required
   Actual Output: Status 400, error indicates missing domain
   Result: Pass

3. `test_create_library_invalid_domain`
   Initial State: N/A

Input: POST request with invalid UUID for domain
Expected Output: HTTP 400 Bad Request; invalid domain
Actual Output: Status 400, error indicates domain field issue
Result: Pass

4. `test_create_library_success_sets_pending_and_returns_task_id`
   Initial State: N/A
   Input: POST request with valid domain and library payload; enqueue analysis patched to return "task-123"
   Expected Output: Library created, analysis status pending, task ID returned
   Actual Output: Library created, `analysis_status = PENDING`, `task_id = "task-123"`
   Result: Pass

5. `test_create_library_enqueue_raises_marks_failed_and_task_id_none`

   Initial State: N/A
   Input: POST request with valid payload; enqueue analysis patched to raise RuntimeError("boom")
   Expected Output: Library created, analysis status FAILED, task ID is None
   Actual Output: Library `analysis_status = FAILED`, `task_id = None`, error message "boom"
   Result: Pass

6. `test_patch_library_success`
   Initial State: Library "OldName" exists under a domain
   Input: PATCH request to update `library_name` to "NewName"
   Expected Output: Library updated successfully; name changed
   Actual Output: `library_name = "NewName"` confirmed in response and database
   Result: Pass

7. `test_put_library_success`
   Initial State: Library "Name1" exists
   Input: PUT request with new domain, name, URL, and programming language
   Expected Output: Library updated successfully with all new fields

Actual Output: Fields updated as expected in response and database
Result: Pass

8. `test_put_library_rejects_non_github_url`
   Initial State: Library exists with a GitHub URL
   Input: PUT request with a non-GitHub URL
   Expected Output: HTTP 400 Bad Request; URL field rejected
   Actual Output: Status 400; error returned for URL
   Result: Pass

9. `test_delete_library_not_found`
   Initial State: N/A
   Input: DELETE request with non-existent library ID
   Expected Output: HTTP 404 Not Found
   Actual Output: Status 404
   Result: Pass

10. `test_delete_library_success`
    Initial State: Library "Temp" exists
    Input: DELETE request to remove the library
    Expected Output: HTTP 204 No Content; library deleted from database
    Actual Output: Status 204; library no longer exists in database
    Result: Pass

11. `test_library_metric_value_update_requires_metrics_dict`
    Initial State: Library "A" exists
    Input: POST request with invalid metrics payload (list instead of dict)
    Expected Output: HTTP 400 Bad Request; error indicates metrics must be dict
    Actual Output: Matches expected
    Result: Pass

12. `test_library_metric_value_update_creates_value_and_evidence`
    Initial State: Library "A" exists; metrics "Stars Count" and "Forks Count" exist
    Input: POST request updating metrics with values and evidence; includes unknown metric
    Expected Output: Values and evidence updated for known metrics; unknown metric ignored; HTTP 200

Actual Output: Matches expected; database updated correctly
Result: Pass

13. `test_metric_value_bulk_update_rejects_non_list`
    Initial State: N/A
    Input: POST request with non-list payload
    Expected Output: HTTP 400 Bad Request; error indicates expected a list
    Actual Output: Matches expected
    Result: Pass

14. `test_metric_value_bulk_update_empty_list_returns_200`
    Initial State: N/A
    Input: POST request with empty list
    Expected Output: HTTP 200 OK; status indicates no data to update
    Actual Output: Matches expected
    Result: Pass

15. `test_metric_value_bulk_update_success`
    Initial State: Library "A" exists; metrics "Stars Count" and "Forks Count" exist
    Input: POST request with multiple update entries including empty and None values
    Expected Output: HTTP 200 OK; values updated accordingly; unknown library skipped
    Actual Output: Matches expected; stars updated to 99, forks set to None
    Result: Pass

5.2 User Management Module

1. `test_list_users_superadmin`
    Input: Superadmin requests list of users
    Expected Output: HTTP 200 OK with serialized user list
    Actual Output: Response status is either 200 or 404 depending on endpoint availability
    Result: Pass

2. `test_list_users_admin_forbidden`
   Input: Admin user attempts to access user list endpoint
   Expected Output: HTTP 403 Forbidden
   Actual Output: Response status is either 403 or 404 depending on endpoint availability
   Result: Pass

3. `test_list_users_unauthenticated`
   Input: Unauthenticated request to list users endpoint
   Expected Output: HTTP 401 Unauthorized
   Actual Output: Response status is either 401 or 404 depending on endpoint availability
   Result: Pass

4. `test_update_user_basic_fields`
   Input: Superadmin updates another user's first and last name
   Expected Output: HTTP 200 OK with updated user fields
   Actual Output: Response status is either 200 or 404 depending on endpoint availability
   Result: Pass

5. `test_admin_can_update_user_but_not_role`
   Input: Admin attempts to update a user's first name and role
   Expected Output: First name updated but role change ignored
   Actual Output: Response 200 OK and role remains unchanged
   Result: Pass

6. `test_user_can_update_own_profile`
   Input: User updates their own first name
   Expected Output: HTTP 200 OK and first name updated
   Actual Output: Response 200 OK and first name modified
   Result: Pass

7. `test_admin_cannot_edit_other_user_basic_info`
   Input: Admin attempts to change another user's first name
   Expected Output: Change is rejected and original value remains
   Actual Output: User's first name remains unchanged
   Result: Pass

8. `test_update_user_unauthenticated`
   Input: Unauthenticated user attempts to update user profile
   Expected Output: HTTP 401 Unauthorized
   Actual Output: Response status is either 401 or 404 depending on endpoint availability
   Result: Pass

9. `test_user_cannot_change_others_password`
   Input: Regular user attempts to change another user's password
   Expected Output: HTTP 403 Forbidden
   Actual Output: Response status 403 Forbidden
   Result: Pass

10. `test_superadmin_can_change_others_password`
    Input: Superadmin changes another user's password
    Expected Output: HTTP 200 OK and `set_password()` called
    Actual Output: Password updated and response 200 OK
    Result: Pass

11. `test_owner_can_change_own_password`
    Input: User changes their own password with correct old password
    Expected Output: HTTP 200 OK and password updated
    Actual Output: `set_password()` called and response 200 OK
    Result: Pass

5.3 Automated Metrics Module

1. `test_extract_repo_info_valid_url`
   Input: Valid GitHub URL
   Expected Output: Repo owner and name parsed correctly
   Actual Output: Matches expected
   Result: Pass

2. `test_extract_repo_info_strips_git_suffix`
   Input: URL ending with .git
   Expected Output: Repo owner and name parsed without .git
   Actual Output: Matches expected
   Result: Pass

3. `test_extract_repo_info_invalid_url_raises`
   Input: Invalid GitHub URL
   Expected Output: Raises ValueError
   Actual Output: Matches expected
   Result: Pass

4. `test_parse_last_page_from_link_returns_page_number`
   Input: GitHub Link header with last page
   Expected Output: Returns last page number
   Actual Output: Matches expected (42)
   Result: Pass

5. `test_parse_last_page_from_link_none_or_missing_last_returns_none`

   Input: Empty, None, or Link header missing last
   Expected Output: Returns None
   Actual Output: Matches expected
   Result: Pass

6. `test_search_count_uses_search_endpoint_and_reads_total_count`
   Input: GitHub search query
   Expected Output: Calls /search/issues; returns total_count
   Actual Output: Matches expected (123)
   Result: Pass

7. `test_get_commits_past_five_years_returns_0_on_409`
   Input: GitHub returns 409
   Expected Output: Returns 0
   Actual Output: Matches expected
   Result: Pass

8. `test_get_commits_past_five_years_uses_last_page_if_present`
   Input: Link header includes last page
   Expected Output: Returns last page number
   Actual Output: Matches expected (77)
   Result: Pass

9. `test_get_commits_past_five_years_falls_back_to_json_length`
   Input: Link header missing
   Expected Output: Returns length of commits array

Actual Output: Matches expected (1)
Result: Pass

10. `test_get_file_type_counts_counts_text_and_binary`
    Input: Repo tree with text and binary files
    Expected Output: Returns counts of text and binary files
    Actual Output: Matches expected (text=2, binary=2)
    Result: Pass

11. `test_get_file_type_counts_returns_0_0_on_non_200`
    Input: Response status not 200
    Expected Output: Returns (0,0)
    Actual Output: Matches expected
    Result: Pass

12. `test_get_github_api_metrics_happy_path`
    Input: Repo with metrics available; patched internal methods
    Expected Output: Returns dictionary with all GitHub metrics
    Actual Output: Matches expected
    Result: Pass

13. `test_get_github_api_metrics_wraps_request_exception`
    Input: github_get raises RequestException
    Expected Output: Raises Exception with "GitHub API Error"
    Actual Output: Matches expected
    Result: Pass

14. `test_analyze_repo_filters_to_target_int_metrics`
    Input: Metrics include int, None, string
    Expected Output: Only int metrics retained
    Actual Output: Matches expected
    Result: Pass

15. `test_run_analysis_and_get_data_returns_repo_name_and_metric_data`

    Input: Patched _analyze_repo returning metrics
    Expected Output: Returns repo_name and metric_data
    Actual Output: Matches expected
    Result: Pass

16. `test_analyze_repo_task_success_updates_metrics_and_marks_success`

    Input: Valid library, metrics, RepoAnalyzer returns metrics
    Expected Output: Updates numeric metrics, marks library analysis as SUCCESS
    Actual Output: Matches expected
    Result: Pass

17. `test_analyze_repo_task_sets_running_status_early`
    Input: Library under analysis
    Expected Output: Analysis status is set to RUNNING before metrics updated
    Actual Output: Matches expected
    Result: Pass

18. `test_analyze_repo_task_exception_marks_failed_and_raises`
    Input: RepoAnalyzer raises RuntimeError
    Expected Output: Library marked as FAILED, analysis_error recorded, exception propagated
    Actual Output: Matches expected
    Result: Pass

19. `test_analyze_repo_gitstats_task_success_sets_report_and_metric`

    Input: Valid library, RepoAnalyzer returns ok
    Expected Output: GitStats paths set, library gitstats_status SUCCESS, metric created
    Actual Output: Matches expected
    Result: Pass

20. `test_analyze_repo_gitstats_task_success_without_metric_still_succeeds`

    Input: Valid library, no GitStats metric exists
    Expected Output: Task succeeds, library gitstats_status SUCCESS, no metric created
    Actual Output: Matches expected
    Result: Pass

21. `test_analyze_repo_gitstats_task_exception_marks_failed_and_raises`

Input: RepoAnalyzer.run_gitstats_only raises RuntimeError
Expected Output: Library gitstats_status FAILED, error recorded, exception propagated
Actual Output: Matches expected
Result: Pass

22. `test_analyze_library_success_202`
    Initial State: Library "A" exists
    Input: POST request to analyze library; enqueue patched to return task IDs
    Expected Output: HTTP 202 Accepted; analysis queued with task IDs
    Actual Output: Status 202; response includes analysis_task_id "t1" and gitstats_task_id "g1"
    Result: Pass

23. `test_analyze_library_enqueue_returns_none_400`
    Initial State: Library "A" has `analysis_error = "bad"`
    Input: POST request to analyze library; enqueue patched to return None
    Expected Output: HTTP 400 Bad Request; error returned
    Actual Output: Matches expected
    Result: Pass

24. `test_analyze_library_enqueue_raises_500_and_marks_failed`
    Initial State: Library "A" exists
    Input: POST request; enqueue patched to raise RuntimeError("boom")
    Expected Output: HTTP 500 Internal Server Error; library analysis status FAILED and error recorded
    Actual Output: Matches expected
    Result: Pass

25. `test_analyze_domain_libraries_mixed_results`
    Initial State: Domain with libraries "A" and "B"
    Input: POST request to analyze all domain libraries; enqueue patched to succeed for "A" and fail for "B"
    Expected Output: HTTP 202 Accepted; queued and failed lists populated appropriately
    Actual Output: Matches expected; queued includes "A", failed includes

"B"
Result: Pass

26. `test_analyze_domain_libraries_exception_marks_failed`
    Initial State: Domain with libraries "A" and "B"
    Input: POST request; enqueue raises exception for "A"
    Expected Output: Library "A" marked FAILED; "B" queued; response HTTP 202
    Actual Output: Matches expected
    Result: Pass

27. `test_library_analysis_missing_url`
    Input: Library with no URL
    Expected Output: Task not queued, library marked as FAILED, error set
    Actual Output: Matches expected
    Result: Pass

28. `test_library_analysis_with_url_queues_both_tasks_and_sets_ids`
    Input: Library with valid URL
    Expected Output: Both Celery tasks queued, library task IDs recorded, analysis and gitstats statuses set to PENDING
    Actual Output: Matches expected
    Result: Pass

5.4 Domains Page Module

1. `test_create_domain_success`
    Initial State: Create user and fixture of API client
    Input: Create payload containing domain name, description, and `creator_ids` with the creator and call POST
    Expected Output: Response returns status 201 for created, and querying for the domain object contains the same as in the payload
    Actual Output: Matches expected
    Result: Pass

2. `test_create_domain_missing_domain_name`
    Initial State: Create fixture of API client
    Input: Create payload with only description and call POST

Expected Output: Response returns 400 bad request and informs of missing field
Actual Output: Matches expected, data returns dictionary with `domain_name` and value of "This field is required"
Result: Pass

3. `test_create_domain_missing_description`
   Initial State: Create fixture of API client
   Input: Create payload with only domain name and call POST
   Response returns 400 bad request and informs of missing field
   Actual Output: Matches expected, data returns dictionary with "description" and value of "Description field is required"
   Result: Pass

4. `test_list_domains`
   Initial State: Create fixture of API client and create two domain objects
   Input: Calling GET for domain
   Expected Output: Response 200 OK with list of domains matching the created domains
   Actual Output: Matches expected
   Result: Pass

5. `test_update_domain_sets_creators`
   Initial State: Create domain object with only `domain_name` and description and fixture of API client
   Input: Create two users and add them to a payload with updated domain name and description. Then call POST with the domain ID
   Expected Output: Response 200, domain should contain the new information and be linked to the two users
   Actual Output: Matches expected
   Result: Pass

6. `test_delete_domain`
   Initial State: Create fixture of API client and create a domain object
   Input: Call DELETE with the domain ID
   Expected Output: Querying for the domain ID returns None
   Actual Output: Matches Expected
   Result: Pass

5.5 Comparison Module

1. `test_ahp_calculations_success`
   Initial State: Domain "AHP Test Domain" with category weight "Performance": 1.0; Metric in category "Performance"; Library "TestLib"
   Input: GET request to `/api/ahp-calculations/{domain_ID}/` with mocked rules.json and categories.json
   Expected Output: HTTP 200 OK; response includes domain name, global ranking containing "TestLib"; library's ahp_results saved with positive overall score
   Actual Output: library.ahp_results["overall_score"] is greater than 0
   Result: Pass

2. `test_ahp_calculations_not_found`
   Initial State: N/A
   Input: GET request to `/api/ahp-calculations/999/` (non-existent domain ID)
   Expected Output: HTTP 404 Not Found
   Actual Output: Status 404 returned
   Result: Pass

3. `test_ahp_calculations_with_weighted_categories`
   Initial State: Domain "Weighted Domain" with category weights "Performance": 0.8, "Security": 0.2; Metrics for both categories; Library "SecureLib"
   Input: GET request to `/api/ahp-calculations/{domain_ID}/` with mocked rules.json and categories.json
   Expected Output: HTTP 200 OK; response includes both "Performance" and "Security" in category_details; library's ahp_results include scores for both categories
   Actual Output: library.ahp_results["category_scores"] contains both categories
   Result: Pass

4. `test_ahp_calculations_basic`
   Initial State: Domain with libraries "A" and "B"; metric "Score" exists; rules.json and categories.json mocked
   Input: GET request to AHP calculation endpoint
   Expected Output: HTTP 200 OK; global ranking includes both li-

braries; ahp_results populated for both
Actual Output: Matches expected; overall_score and category_scores present
Result: Pass

5. `test_domain_comparison_returns_metrics_and_rows_with_values_and_evidence`

   Initial State: Domain with libraries "A" and "B"; metrics "Stars Count" and "Forks Count"; metric values assigned
   Input: GET request to domain comparison endpoint
   Expected Output: HTTP 200 OK; metrics and library rows include values and evidence; gitstats_report_url populated
   Actual Output: Matches expected
   Result: Pass

5.6 Visualization Module

1. `test_library_raw_values_appear`
   Initial State: With a new domain, create library "A" and library "B", and input "123" and "456" for the Commit Count metric for each library respectively.
   Input: Selecting the two libraries and associated commit count metric and press visualize.
   Expected Output: Bar graph of Commit Count appears with library "A" and "B" as a bar graph, with the value of the bar being "123" and "456" respectively.
   Actual Output: Matches expected
   Result: Pass

2. `test_missing_metric_values`
   Initial State: With a new domain, create library "A" and library "B", and input "123" for "A" for the Commit Count metric.
   Input: Selecting the two libraries and associated commit count metric and press visualize.
   Expected Output: No graph appears, error message appears informing user that an issue occurred.
   Actual Output: No graph appears, error message stating "Invalid metrics selected" appears above the visualization button.
   Result: Pass

3. `test_error_shown_when_less_then_two_libraries_are_selected`
   Initial State: With a new domain, create library "A" and library "B", and input "123" for "A" for the Commit Count metric.
   Input: Select "A" and Commit Count metric and press visualize.
   Expected Output: No graph appears, error message appears informing user that an issue occurred.
   Actual Output: No graph appears, error message stating "At least two libraries must be selected" appears above the visualization button.
   Result: Pass

4. `test_only_display_nontext_metrics`
   Initial State: With a new domain, create a new metric called Institutions and set the type to string. Create library "A" and library "B" and metric values "McMaster" and "UofT" for the metric respectively.
   Input: Select "A" and "B".
   Expected Output: The Institutions metric should not appear in the metrics tab.
   Actual Output: Matches expected.
   Result: Pass

5. `test_download_visualizations`
   Initial State: With a new domain, create library "A" and library "B", and input "123" and "456" for the Commit Count metric for each library respectively.
   Input: Select both libraries and the Commit Count metric and press Visualize, then select Download All.
   Expected Output: A zip folder with the visualization is downloaded on the user's computer. Unzipping and opening the folder we can see the same visualized graph.
   Actual Output: Matches expected.
   Result: Pass

5.7 Database Persistence Module

1. `test_domain_str_returns_name`
   Initial State: N/A
   Input: Directly creating a new domain object with `domain_name` "Alpha" and description "desc"

Expected Output: Calling newly created object returns the domain name "Alpha"
Actual Output: Returned "Alpha"
Result: Pass

2. `test_get_domain_ID_returns_string_uuid`
   Initial State: N/A
   Input: Directly creating a new domain object with `domain_name` "Beta" and description "desc"
   Expected Output: Calling `get_domain_ID()` on new domain returns the UUID
   Actual Output: Method matches `domain_id` parameter and is instance of string
   Result: Pass

3. `test_defaults_and_blank_fields`
   Initial State: N/A
   Input: Directly creating a new domain object with `domain_name` "Gamma"
   Expected Output: Initializes domain object with default fields `published: False, paper_name: None, paper_url: "", category_weights:`
   Actual Output: Output matches expected
   Result: Pass

4. `test_unique_domain_name_constraint`
   Initial State: Domain object with `domain_name` "Delta" and description "one"
   Input: Directly creating a new domain object with `domain_name` "Delta" and description "two"
   Expected Output: Throws integrity error due to duplicate domain names
   Actual Output: Matches expected
   Result: Pass

5. `test_creators_many_to_many_assignment`
   Initial State: Create two users with admin role
   Input: Create two domain objects with separate names, add both creators to each domain.
   Expected Output: Filtering in each domain's creators by the id of the creator, both creators should exist

Actual Output: Matches expected
Result: Pass

6. `test_library_str_returns_name`
   Initial State: N/A
   Input: Directly creating a new library object with `library_name` "LibA" under a domain "D1"
   Expected Output: Calling `str()` on the library object returns "LibA"
   Actual Output: Returned "LibA"
   Result: Pass

7. `test_get_library_id_returns_string_uuid`
   Initial State: N/A
   Input: Directly creating a new library object "LibB" under a domain "D2"
   Expected Output: Calling `get_library_id()` returns the library's UUID as a string
   Actual Output: Method returns a string that matches the library_ID
   Result: Pass

8. `test_defaults_and_blank_fields`
   Initial State: N/A
   Input: Directly creating a new library object "LibC" under a domain "D3"
   Expected Output: Default fields initialized: `ahp_results=`, optional fields None, statuses set to pending, timestamps set
   Actual Output: Matches expected default values
   Result: Pass

9. `test_unique_library_name_per_domain_constraint`
   Initial State: Library object "SameName" exists under domain "D4"
   Input: Attempt to create another library with `library_name="SameName"` in the same domain
   Expected Output: Raises `IntegrityError` due to duplicate name in the same domain
   Actual Output: IntegrityError raised
   Result: Pass

10. `test_same_library_name_allowed_in_different_domains`
    Initial State: Two different domains "D5A" and "D5B"

Input: Create libraries with the same name "SharedName" in both domains
Expected Output: Each domain can have the same library name independently
Actual Output: Both libraries exist in their respective domains
Result: Pass

11. `test_unique_url_per_domain_constraint`
    Initial State: Library object with URL "https://github.com/x/y" exists in domain "D6"
    Input: Attempt to create another library with the same URL in the same domain
    Expected Output: Raises `IntegrityError` due to duplicate URL
    Actual Output: IntegrityError raised
    Result: Pass

12. `test_same_url_allowed_in_different_domains`
    Initial State: Two different domains "D7A" and "D7B"
    Input: Create libraries with the same URL "https://github.com/x/y" in both domains
    Expected Output: URL uniqueness is enforced per domain only
    Actual Output: Both libraries exist in their respective domains
    Result: Pass

13. `test_null_url_does_not_break_basic_creation`
    Initial State: N/A
    Input: Create a library with `url=None` under domain "D8"
    Expected Output: Library created successfully with `url=None`
    Actual Output: Library created, `url` is None
    Result: Pass

14. `test_library_metric_value_str_format`
    Initial State: Library "RepoA" and Metric "Stars Count" exist
    Input: Create a LibraryMetricValue with `value={"count": 10}`
    Expected Output: `str(lmv)` returns "RepoA - Stars Count: {'count': 10}"
    Actual Output: Returned string matches expected
    Result: Pass

15. `test_value_id_is_uuid`
    Initial State: Library "RepoA" and Metric exist
    Input: Create a LibraryMetricValue with numeric value 5
    Expected Output: `value_ID` is an instance of UUID
    Actual Output: Matches expected
    Result: Pass

16. `test_value_can_be_null`
    Initial State: Library "RepoA" and Metric exist
    Input: Create a LibraryMetricValue with `value=None`
    Expected Output: Value remains None; string representation shows "None"
    Actual Output: Matches expected
    Result: Pass

17. `test_optional_fields_defaults`
    Initial State: Library and Metric exist
    Input: Create a LibraryMetricValue without providing optional fields
    Expected Output: `value, evidence, collected_by = None`; `last_modified` is set
    Actual Output: Matches expected defaults
    Result: Pass

18. `test_last_modified_updates_on_save`
    Initial State: LibraryMetricValue created with value 1
    Input: Modify value to 2 and save
    Expected Output: `last_modified` timestamp updated
    Actual Output: `last_modified` ¿= previous timestamp
    Result: Pass

19. `test_cascade_delete_when_library_deleted`
    Initial State: Library "ToDelete" exists with one associated LibraryMetricValue
    Input: Delete the library
    Expected Output: Associated LibraryMetricValue is deleted
    Actual Output: LibraryMetricValue no longer exists
    Result: Pass

20. `test_cascade_delete_when_metric_deleted`
    Initial State: Metric "Forks Count" exists with one LibraryMetricValue

Input: Delete the metric
Expected Output: Associated LibraryMetricValue is deleted
Actual Output: LibraryMetricValue no longer exists
Result: Pass

21. `test_str_returns_email`
    Input: User with email "alice@example.com"
    Expected Output: (str(user)) returns the email
    Actual Output: Matches expected
    Result: Pass

22. `test_get_full_name_returns_none_if_missing_parts`
    Input: User without first_name and last_name
    Expected Output: (get_full_name()) returns None
    Actual Output: Matches expected
    Result: Pass

23. `test_get_full_name_concatenates_first_and_last`
    Input: User with first_name "Carol" and last_name "Danvers"
    Expected Output: (get_full_name()) returns "Carol Danvers"
    Actual Output: Matches expected
    Result: Pass

24. `test_role_default_is_user`
    Input: User created without specifying role
    Expected Output: "role" defaults to "user"
    Actual Output: Matches expected
    Result: Pass

25. `test_email_unique_constraint`
    Input: Attempt to create two users with the same email
    Expected Output: Raises IntegrityError
    Actual Output: Matches expected
    Result: Pass

6 Changes Due to Testing

6.1 AHP Methodology Alignment

Following the feedback from the Rev 0 meeting, we identified a discrepancy between our initial implementation and the established research methodology.

- **Previous Iteration:** AHP was only applied to calculate the "Local Weights" of libraries within a single criterion. Users were then asked to manually assign a flat percentage to each overarching criterion.
- **The Revision:** To align with the formal AHP framework, we transitioned to a Pairwise Comparison Matrix for criterion weighting. Users now compare the relative importance of categories (e.g., Usability vs. Reliability) using the Saaty 1–9 scale. The system then mathematically derives the global weights and validates the results via a Consistency Ratio (CR) check, ensuring the data is scientifically sound.

6.2 Visualization and Data Density

We addressed the limitations regarding how library rankings were displayed to improve transparency.

- **Graph Scalability:** Previously, the dashboard was limited to the top 10 libraries to avoid overcrowding. However, to provide a comprehensive view of the domain, we have redesigned the visualization format to clearly accommodate and display all libraries simultaneously.
- **Tabular Data:** Based on user recommendations, we have introduced a Ranking Table to complement the graphical data. This allows users to view precise numerical scores and rankings alongside the visual trends.

6.3 Data Integrity and Input Validation

To reduce user error, we refined the data entry process. We replaced open-ended text fields with context-aware inputs. The system now provides drop-down menus for Booleans and Ranges, preventing invalid data types from corrupting the AHP calculations. *Note: No changes were required based on current unit testing results, as the core logic remains stable.*

7 Automated Testing

Pytest and the `pytest-django` ecosystem serve as the foundation for our automated testing suite, which offers a high-performance, adaptable framework for verifying environment setups and application logic. To ensure that model-level constraints are carefully enforced before code reaches production, we use specialized decorators like `@pytest.mark.django_db` to safely communicate with a temporary test database. We can swiftly find and fix errors in our fundamental business logic, such the concatenation rules for user profiles, by utilizing Pytest's sophisticated assertion introspection.

Using Django's `SimpleTestCase` and the `unittest.mock` module, we developed a strong configuration testing method in addition to standard data validation. This enables us to test that our settings dynamically swap between SQLite and MySQL engines as planned by programmatically simulating various deployment scenarios (e.g., Local vs. Production). We make sure the application stays stable and properly configured across different infrastructure states without having to manually review environment files by simulating environment variables and altering settings within our tests.

8 Trace to Requirements

Table 16 presents the traceability between the system test cases and the requirements defined in the Software Requirements Specification (SRS). The table indicates which test cases provide verification evidence for each requirement implemented in the system.

Table 14: Traceability Between Requirements and System Test Cases

Requirement	T-AC-1	T-AC-2	T-AC-3	T-AC-4	T-AC-5	T-DM-1	T-DM-2	T-DM-3	T-DM-4	T-DM-5	T-VR-1	T-VR-2	T-VR-3	T-VR-4	T-VR-5	T-VR-6
FR2	X															
FR3		X	X													
FR4				X												
FR5						X										
FR7							X	X								
FR10							X									
FR11																X
FR12								X			X					
FR13											X	X				
FR14										X						
FR15													X			
SR-AC4					X											
SR-AC5	X	X	X	X												
SR-VIS1											X					
SR-VIS2												X				
PR-RFT1									X						X	
SR-INT5									X						X	
LF-AR2														X		
LF-AR3																X
LF-SR2														X		
UH-AR1														X		
OE-PR4										X			X			

The tests listed in this report were primarily designed to verify functional requirements. However, some of these tests also provide evidence related to certain non-functional requirements, which is why those requirements appear in the traceability matrix. Requirements that were still under development or removed from the final project scope are not included in the table and are discussed earlier in the *Features Currently in Progress* subsection of the *Functional Requirements Evaluation* section.

9 Trace to Requirements (Nonfunctional)

Table 15 summarizes traceability between nonfunctional requirements (NFR-related requirement IDs from the SRS) and the nonfunctional test cases (T–NF1 through T–NF6) defined in the VnV Plan.

Table 15: Nonfunctional Requirements Traceability Matrix

Requirement ID	**T–NF1**	**T–NF2**	**T–NF3**	**T–NF4**	**T–NF5**	**T–NF6**
PR–SL1	X					
PR–RFT1		X				
PR–RFT2		X				
PR–RFT3		X				
SR–INT5		X				
UH–EU1			X			
LF–AR2			X			
UH–UP3			X			
LF–SR2			X			
PR–PA1				X		
FR12				X		
PR–SC1					X	
PR–SC2					X	
PR–SC3					X	
SR–AC6					X	
SR–IM1					X	
SR–AU1					X	
MS–MR1						X
MS–MR2						X
MS–MR3						X
MS–MR4						X
MS–MR5						X
MS–MR6						X

The tests listed in this section were designed to provide verification evidence for the nonfunctional requirements in the SRS. An "X" indicates that the corresponding test case contributes evidence for that requirement. Note that evidence may include Pass, Partial, or Pending outcomes as reported in the nonfunctional requirements evaluation section.

10 Trace to Modules

Table 16 presents the traceability between the system test cases and the software modules defined in the system architecture. The table indicates which test cases provide verification evidence for each module implemented in the system.

Table 16: Traceability Between Modules and System Test Cases

Module	T-AC-1	T-AC-2	T-AC-3	T-AC-4	T-AC-5	T-AC-6	T-AC-7	T-DM-1	T-DM-2	T-DM-3	T-DM-4	T-DM-5	T-VR-1	T-VR-2	T-VR-3	T-VR-4	T-VR-5	T-VR-6
M2																X		
M3					X		X			X		X			X	X		X
M4						X		X	X	X								
M5	X	X					X											
M6			X	X		X		X										
M7	X																	
M8									X									
M9								X										X
M10											X	X			X	X	X	X
M11										X			X					
M12													X	X	X	X		X
M14						X			X	X		X			X			
M15									X									
M16										X			X	X				
M17						X			X	X	X		X				X	
M18											X						X	
M19								X										

11 Code Coverage Metrics

The following report is automatically generated by pytest. We obtained a total coverage of 73%. The actual coverage should be slightly higher since some files that contain no logic are being counted towards the percentage (i.e. edit_history/views).

Table 17: Backend Code Coverage Report

File	Statements	Missing	Excluded	Coverage
src/backend/api/database/backup_logs/admin.py	1	0	0	100%
src/backend/api/database/backup_logs/models.py	12	1	0	92%
src/backend/api/database/backup_logs/views.py	1	1	0	0%
src/backend/api/database/domain/models.py	17	0	0	100%
src/backend/api/database/domain/serializers.py	16	1	0	94%
src/backend/api/database/domain/views.py	51	15	0	71%
src/backend/api/database/edit_history/admin.py	1	0	0	100%
src/backend/api/database/edit_historymodels.pyy	16	1	0	94%
src/backend/api/database/edit_historyviews.py	1	1	0	0%
src/backend/api/database/libraries/models.py	38	0	0	100%
src/backend/api/database/libraries/serializers.py	35	2	0	94%
src/backend/api/database/libraries/views.py	67	1	0	99%
src/backend/api/database/library_metric_values/admin.py	8	0	0	100%
src/backend/api/database/library_metric_values/models.py	14	0	0	100%

File	Statements	Missing	Excluded	Coverage
src/backend/api/database/library_metric_values/serializers.py	12	0	0	100%
src/backend/api/database/library_metric_values/views.py	153	7	0	95%
src/backend/api/database/metrics/admin.py	6	0	0	100%
src/backend/api/database/metrics/models.py	15	1	0	93%
src/backend/api/database/metrics/serializers.py	10	0	0	100%
src/backend/api/database/metrics/views.py	53	28	0	47%
src/backend/api/database/services.py	241	129	0	46%
src/backend/api/models.py	5	0	0	100%
src/backend/api/serializers.py	4	4	0	0%
src/backend/api/services/github_http.py	67	0	0	100%
src/backend/api/tasks.py	109	19	0	83%
src/backend/api/utils/analysis.py	26	0	0	100%
src/backend/manage_local.py	55	55	0	0%
src/backend/users/auth.py	11	5	0	55%
src/backend/users/backends.py	15	15	0	0%
src/backend/users/models.py	14	0	0	100%
src/backend/users/permissions.py	7	7	0	0%
src/backend/users/serializers.py	28	5	0	82%

File	Statements	Missing	Excluded	Coverage
`src/backend/users/views/auth\_views.py`	131	32	0	76%
`src/backend/users/views/profile.py`	25	12	0	52%
Total	1265	342	0	73%

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

   Haniye: One aspect that went well while writing this deliverable was that many of the test scenarios were already clear from the development process. While implementing features such as domain management, metric editing, and visualization, the system was frequently tested to verify correct behaviour. As a result, when preparing the VnV report it was easier to define the test inputs and expected outcomes because the implemented features had already been validated during development.
   Fei: I was responsible for the unit test portion. Since we decided to write unit test as we develop, instead of all at the end, writing this portion was much simpler as I just had to compile what we had into a report format.

   Fei: I was responsible for the unit test portion. Since we decided to write unit test as we develop, instead of all at the end, writing this portion was much simpler as I just had to compile what we had into a report format.

Ghena: A lot of test cases were already written and used throughout the development process, making sure that any ant changes to the code follows the logic of the requirements. This made the document easy to write since a lot of it was alreadt built.

Awurama: Converting our real testing into a consistent structure (summary to method to results to analysis to next steps) worked well. We reused concrete evidence (timings, UI observations, CI output, and code excerpts) and traceability was straightforward because the VnV Plan already mapped each T–NF test to requirement IDs.

2. What pain points did you experience during this deliverable, and how did you resolve them?

Haniye: A minor challenge during this deliverable was ensuring that the tests described in the report aligned with the original VnV Plan while also reflecting the current implementation of the system. Some features had changed during development, which made it necessary to adjust how certain tests were documented. Feedback from the teaching assistant helped clarify how the tests should be structured, allowing the report to accurately represent the implemented functionality.

Fei: Setting up the testing frameworks was complete, due to the way our project repository was structured. Our code also wasn't as modularized as intended, so many unit tests had to have a lot of mocking in place. The way I resolved them was just adding the testing frameworks earlier, so there was more time to deal with unexpected issues, as opposed to having to cram it all in this week.

Ghena: A significant challenge during this deliverable was ensuring the AHP Ranking Module (M11) was implemented correctly to avoid any methodology misunderstandings. To resolve this, the team held a focused meeting with the supervisor to walk through the exact calculation requirements and conducted a thorough review of the assigned research paper.

Awurama: Some planned tests were not feasible in Rev0 (domain filter no-op, HTTP 429 not reproducible, backup/restore requires DB-level procedure). We reported what we could verify, marked missing items

as *Not tested/Pending*, and listed clear next steps. Manual stopwatch timing also adds reaction-time error, so we documented the method and treated results as conservative upper bounds.

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

   Haniye: Some parts of this document were influenced by discussions with the project supervisor. For example, the bulk data upload feature was removed from the project scope after supervisor feedback indicated that it would not be necessary for the final tool. The supervisor also suggested removing the public sign-up feature and instead allowing a Super Admin to create user accounts directly, with users updating their password afterward.

   Fei: N/A

   Ghena: All of the changes made due to testing came from the our client/stackholder meetings. However The specific direct mapping (e.g., tracing T–AC–1 to M5 and M7) didn't stem from those meetings.

   Awurama: The usability section (T–NF3) stems from a proxy user worksheet with task ratings and comments. The remaining NFR sections were primarily tool/code/CI-based verification (performance timing, offline fault injection and recovery, repo-field spot checks, security configuration review, CI maintainability evidence), so they did not require client input in Rev0.

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

Haniye: Most of the verification and validation activities followed the original VnV Plan. Some small differences occurred due to time constraints and changes in project priorities. For example, the bulk data upload feature was removed from the project scope based on supervisor feedback, so its related test was not implemented. Some tests were also influenced by issues discovered during deployment. For instance, while deploying the system, GitStats initially did not run correctly on the server and the analysis process continued for an unexpectedly long time. During debugging, the request timeout was reduced, which allowed us to observe the system's failure message and verify that the application handled the error properly.

In future projects, some of these adjustments could be anticipated by planning additional tests for integration and deployment scenarios.

Fei: The changes I had to make to the unit test portion was the modules. As we developed the tool, we ended up condensing a few modules together since that made more sense during the development. This just meant combining the related unit tests of the combined modules into one. For future projects, I don't think we can mitigate it completely. Since we designed the project before development, during development new insights arises which results in changes to the theoretical plan.

Ghena: The final VnV activities deviated from the plan primarily because the AHP Calculation Module (M11) required restructuring to align with the methodology in the assigned research paper, replacing an initial simplified model. These changes could have been avoided had the module been revised more properly during the initial design phase. Additionally, features like Excel bulk upload (FR9) were removed to prioritize core stability. While some scope reductions are manageable, predicting the future requirements of this project proved difficult and would remain a challenge in future iterations.

Awurama: Rev0 deviated from the plan mainly due to feasibility and environment constraints: performance used available domains (2 and 10 libraries) with manual timing instead of 5/10/20/40 and automated measurement; reliability used offline-mode fault injection since 429 was not reproducible and caching/warnings were not observable; computational accuracy captured tool priorities plus a GitHub spot check

but did not complete the full manual AHP computation with 2% error checks; security verified settings/auth/logging but did not confirm lockout/session timeout (no explicit configuration found); maintainability used CI evidence but coverage did not meet the planned target and lint automation was not observed. These changes occurred because data, reproducible fault triggers, and infra access were limited in Rev0. In future projects, we can anticipate these risks and plan mitigations (early dataset prep, instrumentation for timing, fault-injection hooks, and explicit Rev0 vs. Rev1 evidence expectations).